

RSS — Technical Build and Management Manual

Surface Deployment Stick — automated Windows 11 deployment for Surface Laptop

RSS — Surface Deployment Stick · Windows 11 deployment for Surface Laptop

Document version	2.0
Date	2026-10-02
Audience	IT administration and workplace management
Classification	Source-available — no open-source license
Generated from	docs/source/ + config/sources.json

Source-available administration document — no open-source license.
This PDF is generated; change the source in docs/source and rebuild with tools/Build-Documentation.ps1.

Contents

1. Purpose, status and reading guide	3
1.1 Status and validation levels	3
1.2 Supported hardware	3
1.3 CORE WARNING	3
2. Architecture at a glance	4
2.1 Separation of responsibilities	4
2.2 Design principles	4
3. Repository structure	4
4. Requirements	5
5. Windows, ADK and WinPE base	5
5.1 Recorded versions	5
5.2 WinPE composition	6
5.3 Boot chain	6
6. Surface driver sources	6
7. Model detection and preflight	6
7.1 Detection sources	6
7.2 Decision order (RSS-SafetyLib)	6
7.3 SystemSKU allowlists	7
8. Driver strategy	7
9. Deployment flow (runtime)	7
10. Disk safety	8
11. Windows servicing	8
12. Offline driver injection	8
13. OOB, Autopilot and network	9
13.1 Expected first-boot behavior	9
13.2 The five OOB error classes	9
13.3 Diagnostic tool	9
14. Building and refreshing media	10
15. Validation	10
16. Physical testing	11
17. Production release	11
18. Troubleshooting (evidence-based)	11
19. Recovery	12
20. Logging	13
21. Checksums and integrity	13
22. Release management	13
23. Security and privacy	13
24. Known limitations	13
Appendix A — Versions and sources	14
Appendix B — Model matrix with validation status	14

1. Purpose, status and reading guide

RSS (Surface Deployment Stick) is a fully automated, bootable Windows 11 deployment stick for Microsoft Surface Laptop 4, 5, 6, 7 and 8 (only the variants named in this document). The stick is designed for situations in which an administrator deliberately boots from USB and needs no input device or interaction.

In one sentence: boot from USB; RSS detects the model, SKU and processor, validates source and target disk including file hashes, wipes Disk 0 only after a countdown of 15 seconds, applies the correct Windows image, injects the complete official Surface driver package and creates the UEFI boot files.

This manual describes release **2.0.0** with production media as of **2026-09-23** (Windows build 26200.9457).

1.1 Status and validation levels

RSS defines four explicit validation levels that must not be confused with one another:

Level	Meaning	Current status
Code validated	CI, syntax and guardrail tests passed	yes, per commit (CI)
Media validated	Validate-RSSMedia.ps1 green on a built stick	yes, on the production stick as of 2026-09-23
Physically validated	full deployment + first boot on physical Surface hardware	only SF6 (field test 2026-09)
Production approved	released tag + checksums + test evidence	tag rss-2026-09-sf8

Models without a physical test are explicitly **NOT PHYSICALLY VALIDATED** (see Appendix B). Software tests never prove that a Surface model deploys correctly on physical hardware.

1.2 Supported hardware

Profile	Model	Platform	INF	Status
SF4	Surface Laptop 4 (AMD)	AuthenticAMD	78	NOT PHYSICALLY VALIDATED
SF5	Surface Laptop 5 (Intel, consumer + for Business)	GenuineIntel	112	NOT PHYSICALLY VALIDATED
SF6	Surface Laptop 6 for Business (Intel)	GenuineIntel	116	physically validated (2026-09)
SF7	Surface Laptop for Business 7th Edition with Intel	GenuineIntel	119	NOT PHYSICALLY VALIDATED
SF8	Surface Laptop for Business 8th Edition with Intel (NOT Snapdragon/ARM)	GenuineIntel	120	NOT PHYSICALLY VALIDATED

The complete SystemSKU list per profile is in [config/sources.json](#); Appendix B contains the summary. Known explicit refusals: Surface Laptop 4 Intel, Surface Laptop 7th Edition Snapdragon, Surface Laptop 5G for Business 7th Edition (SKU [2119](#)) and Surface Laptop 8th Edition Snapdragon. These models result in a safe stop before every write action.

1.3 CORE WARNING

After a successful preflight and a countdown of 15 seconds, RSS wipes the entire internal NVMe disk (Disk 0) without a confirmation prompt. Use the stick only on a device whose data may deliberately be erased.

2. Architecture at a glance

```

USB boot (UEFI, MBR stick)
  → ADK WinPE (boot.wim, startnet.cmd)
  → PowerShell: RSS-Deploy.ps1 (+ RSS-SafetyLib.ps1)
    → preflight: manifest, model, SKU, CPU, media, disks, hashes
    → countdown 15 s
    → wipe Disk 0 + GPT (EFI 260 MB, MSR 16 MB, Windows NTFS)
    → DISM /Apply-Image (install.esd index 1)
    → wimlib extraction of the driver archive + Add-WindowsDriver (offline)
    → BCDBoot (UEFI)
    → green screen + automatic reboot
    → first boot: Windows → OOBE (normal, no bypass)
  
```

RSS is deliberately a self-contained deployment environment without interactive Windows Setup and without an unattend file: the device ends up in a completely normal, MDM/Autopilot-ready OOBE.

2.1 Separation of responsibilities

Layer	Responsibility
UEFI / MBR	make the USB discoverable and bootable; start the active FAT32 partition
ADK WinPE	make PowerShell, WMI, Storage and DISM functionality available
RSS-SafetyLib.ps1	pure decision functions: model/SKU detection, media recognition, target disk selection, hash check
RSS-Deploy.ps1	orchestration: preflight, disk layout, imaging, drivers, boot, logging, visual phases
config/sources.json	single source of versions, models, SKUs, hashes and INF counts; travels on the stick
Images partition	five Windows images, five driver archives, WinPE drivers, manifest, documentation, log
Target NVMe	after full validation, the GPT target for EFI, MSR and Windows

2.2 Design principles

- **Proof before change:** deployment logic changes only with a demonstrable reason, test coverage and a media rebuild.
- **UNKNOWN/AMBIGUOUS/UNSUPPORTED/MISSING/INVALID = STOP:** every uncertain situation is a red safe stop before the first write action.
- **Single source of truth:** versions, models, SKUs, hashes and INF counts live only in `config/sources.json`; README, validation and documentation are derived from it.
- **No internet during deployment:** RSS-Deploy never uses the network; only the maintenance script performs downloads.

3. Repository structure

```

├── .github/           CI workflow, issue and PR templates
├── config/
│   ├── sources.json   CANONICAL manifest (versions, models, SKUs, hashes, INF)
│   └── load-orders/   proven WinPE driver load orders per profile (SF4-SF8)
├── docs/
│   ├── source/        CANONICAL documentation sources (Markdown + templates)
│   ├── generated/     generated DOCX/PDF and stick documentation
│   ├── MEDIA-LAYOUT.md, TESTMATRIX.md, PHYSICAL-TEST-QUICKLIST.md
│   └── ...
└── src/winpe/
  
```

```
| | RSS-SafetyLib.ps1    safety library (pure functions, tested)
| | RSS-Deploy.ps1      deployment orchestration (runs in WinPE)
| | Build-WinPE.ps1     boot.wim build from the official ADK source
| | startnet.cmd         wpeinit + starts RSS-Deploy
| | tools/
| | | Update-RSSMedia.ps1    reproducible media refresh (phases)
| | | Validate-RSSMedia.ps1  read-only stick validation
| | | Test-DriverArchives.ps1 trial-extraction test of the driver archives
| | | Build-Documentation.ps1 DOCX/PDF/stickdocs generation
| | | Build-Checksums.ps1    rebuild checksums/SHA256SUMS.txt
| | tests/               Pester guardrail tests (mocked, never touch real disks)
| | checksums/           SHA256SUMS.txt covering all Git files
```

Not in Git: Windows images, ADK/WinPE files, Surface driver packages, extracted drivers, generated WIM/ESD and run logs (see `.gitignore`). These are large, third-party distributed and/or device-specific; `Update-RSSMedia.ps1` + the manifest reproduce everything from official sources.

4. Requirements

Component	Requirements
Build workstation	Windows 10/11 x64 with PowerShell 5.1 or 7+, local administrator rights
ADK	Windows ADK 10.1.26100.9457 + WinPE add-on, default path C:\ADK (an installation path via subst/symlink is allowed)
wimlib	1.14.5 Windows x64 binaries (URL and SHA-256 in the manifest), available at D:\RSS-Build\wimlib
Working directory	at least 40 GB of free space, default D:\RSS-Build
Target medium	64 GB USB 3.x stick (production routing: Samsung), may be wiped completely
Source image	serviced install.wim from the UUP set as recorded in the manifest
Documentation build	pandoc ≥ 3.6 and Typst ≥ 0.15 (regenerable, see chapter 14 and docs/source/README.md)

5. Windows, ADK and WinPE base

5.1 Recorded versions

Component	Version	Checked
Windows edition	Professional, amd64, Netherlands (nl-NL)	—
Windows baseline	UUP set build 26200.9457	2026-10-02: latest stable (non-preview) LCU for 25H2
LCU	KB5129195 (2026-09-14)	current
.NET CU	KB5126052 (2026-09-08)	current
SafeOS DU (WinRE)	KB5124015 (2026-09-08); successor KB5125758 at the next rebuild	see manifest
Deliberately not used	KB5124010 (26200.9550, preview)	—
ADK	10.1.26100.9457 (2026-09)	current
wimlib	1.14.5 (2026-01-30)	current

The end date of base servicing: Home/Pro 2027-10-12, Enterprise/Education 2028-10-10.

5.2 WinPE composition

The boot environment is built entirely from the official ADK winpe.wim (not the ISO boot.wim). Optional components: WinPE-WMI WinPE-NetFX WinPE-Scripting WinPE-PowerShell WinPE-StorageWMI WinPE-DismCmdlets (each with the en-US language pack), scratch space 512 MB. After integrating them, `Build-WinPE.ps1` verifies that PowerShell, the Storage and Dism modules, the DISM provider, both deployment files and wimlib are present, and writes the SHA-256 to `Build-WinPE.status`.

5.3 Boot chain

`startnet.cmd` runs `wpeinit` and then immediately starts `powershell.exe -NoLogo -NoProfile -ExecutionPolicy Bypass -File X:\Deploy\RSS-Deploy.ps1`. No menu, no input device and no choice are required; on an error, the red safety screen remains on screen.

6. Surface driver sources

Profile	Microsoft package	Date	INF	EOL servicing
SF4	SurfaceLaptop4_AMD_Win11_22631_26.072.27963.0.msi	2026-08-14	78	2027-04-15
SF5	SurfaceLaptop5_Win11_22631_26.084.43979.0.msi	2026-09-22	112	2028-10-25
SF6	SurfaceLaptop6forBusiness_Win11_22631_26.082.26400.0.msi	2026-09-22	116	2030-04-09
SF7	SurfaceLaptop7withIntel_Win11_22631_26.081.15509.0.msi	2026-09-22	119	2031-02-18
SF8	SurfaceLaptop8withIntel_Win11_26100_26.083.37379.0.msi	2026-09-11	120	2032-05-19

All MSIs are downloaded from download.microsoft.com (link per profile in the manifest), checked for SHA-256 as well as a valid Authenticode signature from Microsoft Corporation, and then extracted with `msiexec /a`. The INF count is checked exactly against the manifest — a deviating count is always a stop, never an adjustment of the expected count without verification and a manifest update.

Driver and firmware servicing per model ends on the dates listed in the table (source: Microsoft lifecycle). The earliest EOL in the fleet is Surface Laptop 4 (2027-04-15); plan replacement or an image refresh.

7. Model detection and preflight

7.1 Detection sources

- SMBIOS `SystemProductName` and `SystemSKU` from `HKLM:\HARDWARE\DESCRIPTION\System\BIOS`;
- CPU `VendorIdentifier` from `HKLM:\HARDWARE\DESCRIPTION\System\CentralProcessor\0`;
- `Get-PSDrive` for exactly one RSS data partition (recognition paths SF4 + SF8 `install.esd`);
- `Get-Partition` / `Get-Disk` for the physical USB identity and all NVMe disks.

7.2 Decision order (RSS-SafetyLib)

1. Name contains `Snapdragon` → STOP (ARM);
2. strict regex on `SystemProductName` (five profiles) → no match is STOP;
3. CPU vendor check per profile (SF4 = AuthenticAMD, the rest = GenuineIntel) → mismatch is STOP;
4. `SystemSKU` empty → STOP; SKU on the profile's refusal list → STOP; SKU not on the allowlist → STOP with instructions to verify/update the manifest;
5. manifest missing or incomplete → STOP;

6. image or driver archive missing, too small, or hash mismatch → STOP;
7. `dism /Get-WimInfo` on the archive must yield a readable index 1 → otherwise STOP;
8. disk selection: exactly one internal NVMe on Disk 0, media not on it → otherwise STOP;
9. only after all of the above does the countdown of 15 seconds start.

The regexes and messages are deliberately identical to the proven production logic; the newer layers (SKU allowlist, NVMe unambiguity, hash check) are strictly fail-safe and covered by `tests/RSS-Safety.Tests.ps1`.

7.3 SystemSKU allowlists

Each profile contains an explicit list of supported SKUs in the manifest (based on the official Microsoft Surface SystemSKU reference, checked 2026-10-02) and — where applicable — a refusal list. New regional SKU variants are added only after verification against the Microsoft reference, with a manifest update, tests and a new build round.

8. Driver strategy

Two separate driver layers:

1. **Full official package (target):** after Windows has been applied, the complete Microsoft Surface-Update package (all INFs from the model MSI) is injected offline into the DriverStore. As a result, touchscreen, keyboard/Type Cover, touchpad, Wi-Fi, USB, sensors and platform components work before the first boot, without a network.
2. **WinPE minimization (build/recovery material):** `RSSWinPEDrivers\SFx` + `config/load-orders` contain the minimal driver set for WinPE initialization. The active boot.wim deliberately starts without model-specific drivers, because the ADK source with Microsoft inbox drivers initializes NVMe and USB reliably; the load orders are the recorded, proven order for recovery/build use and are checked for consistency by the validation.

The distinction matters: a small set of input drivers is enough to operate WinPE, but it is not the end goal; the offline DriverStore must contain the full package before the first boot.

9. Deployment flow (runtime)

Step	Action	Safety check
0	module initialization + media recognition	exactly one RSS data partition
0b	load the manifest from the stick	manifest present and complete
1	model/SKU/CPU detection	allowlists and refusal lists
2	check paths + files	image, archive, size
3	physical disk resolution	media ≠ Disk 0, exactly one NVMe = Disk 0
4	archive readable (DISM)	index 1
5	hash check image + archive	manifest SHA-256, before the wipe
6	summary + countdown 15 s	cancel = power off the device
7	wipe Disk 0, GPT: EFI 260 MB / MSR 16 MB / Windows NTFS	—
8	DISM /Apply-Image /CheckIntegrity	SYSTEM hive present
9	wimlib extraction of the archive + INF count	matching the exact 116-style count per profile
10	Add-WindowsDriver -Recurse	DriverStore count before/after logged

Step	Action	Safety check
11	bcdboot /f UEFI /l nl-NL	bootmgfw.efi present
12	green screen + automatic reboot	remove the USB while green

The complete course of the run, including all DISM output, DriverStore counts and result codes, is logged to `<Images>:\RSS-ADK-Deploy.log` (chapter 20).

10. Disk safety

The target disk rule is absolute: **only the single internal NVMe disk, always Disk 0 in WinPE, and never the USB medium.** The fail-safe matrix from the manifest:

```
UNKNOWN = STOP - unknown model, empty SystemSKU or unknown SKU
AMBIGUOUS = STOP - zero or multiple candidate data partitions, multiple NVMe disks
UNSUPPORTED = STOP - unsupported model, ARM/Snapdragon, wrong CPU vendor, refused SKU
MISSING = STOP - missing image, archive, manifest or load order
INVALID = STOP - hash mismatch, unreadable archive, wrong INF count, Disk 0 not NVMe
```

- The countdown of 15 seconds is the last chance to cancel: power off the device during the countdown and the wipe does not start.
- After the green screen has appeared, the stick must never remain ahead of the internal SSD in the UEFI boot order; remove the USB at the green screen, otherwise the same destructive deployment can restart.
- Unexpected situations (multiple NVMe disks, NVMe not on Disk 0, USB on Disk 0, SD/USB media as a target disk candidate) lead to a safe stop, never to "just try it".

11. Windows servicing

The installed base is a serviced UUP image: English Professional base 26100.1 + enablement eKB + checkpoint LCU + current LCU + .NET CU, serviced offline with ADK DISM; WinRE is serviced separately with the SafeOS DU (WinRE does not follow the main LCU). The result is `install.esd` per profile (index 1, recovery compression) with the SHA-256 in the manifest.

Rules for a new servicing round:

1. Only the latest **stable** (non-preview) cumulative update; preview KBs stay out of production (currently deliberately: KB5124010).
2. Update the manifest (LCU/SSU/.NET/SafeOS + build + hash) before the rebuild and run the validation against the new values.
3. SafeOS DUs and .NET CUs appear on their own schedule; adopt newer versions at the next rebuild (see `nextRefresh` in the manifest) — do not rebuild solely for a single package.
4. One media build per servicing round; interim "loose" updates on sticks are forbidden (drift).

12. Offline driver injection

`wimlib-imagex apply` temporarily extracts the driver archive (ESD, `--check`) to `<Windows>:\RSS-DriverStage`; the INF count must match the manifest count exactly. Then `Get-WindowsDriver` records the DriverStore count before injection, `Add-WindowsDriver -Path ... -Driver <stage> -Recurse` injects all drivers offline, and the count is

recorded again. A decreasing count is an error; the staging directory is removed afterwards. All DISM output goes to the log.

13. OOB, Autopilot and network

RSS delivers the device **unchanged** to OOB: no unattend, no local account, no network bypass. That is a deliberate design choice: the installation exists precisely to enable Autopilot/Intune enrollment. Product keys, `ms-cxh:localonly`, `BypassNRO` and similar tricks are forbidden in this production environment (`BypassNRO` has moreover been removed by Microsoft from the 26200 line and `ms-cxh:localonly` has been blocked since late 2025; the validation deliberately fails on these patterns in the deployment code).

13.1 Expected first-boot behavior

- The first boot may show one or more automatic restarts; a field test on SF6 showed a temporary reboot cycle of about five minutes that recovered by itself. With repeated short reboots, wait at least 10 minutes before investigating further.
- After OOB, touchscreen, keyboard/trackpad, Wi-Fi and USB must work immediately from the offline DriverStore.
- Autopilot: after the network connection is up, OOB fetches the Autopilot profile; the device must be registered in the tenant (hardware hash + assigned profile) before the deployment.

13.2 The five OOB error classes

Class	Characteristic	First diagnosis
Image failure	installation/boot problem before OOB; recovery environment or reboot loop	C:\Windows\Panther\setupact.log/setuperr.log
Network failure	hangs at the Dutch network check "De verbinding met Microsoft wordt gecontroleerd" ("Checking the connection with Microsoft") or falls back to Wi-Fi	Get-NetConnectionProfile (must show Internet), nslookup dns.msftncsi.com, http://www.msftconnecttest.com/connecttest.txt (must return "Microsoft Connect Test")
Microsoft service failure	network OK, the OOB service returns an error	event log Microsoft-Windows-CloudExperienceHost/Operational, NCSI/Operational
Autopilot/MDM failure	profile hang after the network check; "Something went wrong"	event log ModernDeployment-Diagnostics-Provider/Autopilot (807/815 = not enrolled/no profile; 171/172 = TPM attestation; 100 = waiting for profile), MdmDiagnosticstool.exe -area Autopilot;TPM -cab C:\autopilot.cab
OOB regression	new behavior after a build change	release health page of the build in question; roll the media back to the previous release

For build 26200.9457, no OS known issue affecting OOB is known (status 2026-10-02); a hang at the network check is therefore in the first instance an **environment issue**: an NCSI probe that is blocked (Wi-Fi always uses the HTTP probe to www.msftconnecttest.com), a firewall that blocks Autopilot endpoints (`ztd.dds.microsoft.com`, `login.live.com`, `*.microsoftaia.azure.net`, `time.windows.com` UDP 123), or a device that is not yet registered in Autopilot. The NCSI active probe must never be disabled as a "fix".

13.3 Diagnostic tool

`tools/Collect-RSSOOBEDIag.ps1` is read-only and, in a failing OOB (Shift+F10), collects everything onto the stick: network profiles, NCSI registry and live probes, Autopilot endpoint probes, services,

OOBE registry, CloudExperienceHost/NCSC/NLA/WLAN event logs, Appx status and Panther/OOBE log files. Use this tool before any assumption; in incidents, always refer to the collected folder.

14. Building and refreshing media

The complete process is programmed in `tools/Update-RSSMedia.ps1` (phases: Drivers → Archives → WinPEDrivers → BootWim → Image → Media → Validate). Main properties:

- downloads only from the official URLs recorded in the manifest, with SHA-256 + Authenticode checks and partial-download protection;
- INF counts checked exactly against the manifest; archives are trial-extracted after building;
- boot.wim built completely fresh from the ADK source (including `RSS-SafetyLib.ps1`);
- images serviced and exported per profile; SafeOS DU and .NET CU from `downloads\`;
- the Media phase refuses Disk 0, non-USB media and system disks; partitions WINPE (FAT32, 2 GB, active) and Images (NTFS, remainder);
- manifest and generated stick documentation are placed on the Images partition (`sources.json` is read by RSS-Deploy before the wipe);
- the Validate phase and then `Validate-RSSMedia.ps1` (read-only, 30+ checks) must be green before any physical test.

Brief procedure overview (full text: `docs/generated/RSS_Technical_Build_and_Management_Manual.*` and `tools/Update-RSSMedia.ps1`):

```
# 1. prerequisites present? (ADK, wimlib, manifest up to date)
.\tools\Update-RSSMedia.ps1 -Phase Drivers -WorkingDir D:\RSS-Build
# 2..6 remaining phases, then:
.\tools\Update-RSSMedia.ps1 -Phase Media -UsbDiskNumber <n> -SourceInstallWim D:\RSS-
Build\install-serviced.wim
.\tools\Validate-RSSMedia.ps1
```

Cloning/duplicating: a block-level clone remains the recommended route; give the clone a unique MBR disk signature and verify labels, partition sizes and the SHA-256 hashes of images/archives. Do not proceed if source and target cannot be distinguished unambiguously.

15. Validation

`tools/Validate-RSSMedia.ps1` (**read-only**) checks, among other things: USB identity, partitions and filesystem state, required files, manifest identity (`sources.json` on stick == repo), boot.wim hash, Secure Boot binaries and BCD flags, image identity and DISM metadata (edition/build/language/KBs), driver archive hashes + integrity + exact INF counts, load orders and INF contents, AST syntax of the deployment logic and safety library, guardrail presence (manifest, hash check, disk selection), absence of interaction and MDM/OOBE risk patterns, and manifest consistency of RSS-INFO.txt. Result per line PASS/WARN/FAIL with a final report; FAIL = exit code 1.

`tests/RSS-Safety.Tests.ps1` (**Pester, mocked**) covers the danger matrix: supported model + valid media (proceeds), unknown model, empty/unknown/refused SKU, SF7 5G, SF8 Snapdragon, ARM name, CPU mismatch, no/multiple NVMe, NVMe not on Disk 0, USB on Disk 0, USB = target disk, zero/multiple data partitions, missing archives, hash mismatch and incomplete manifest — all STOP. The tests never touch a real disk and run in CI.

`tools/Test-ManifestConsistency.ps1` checks manifest structure, hash format, URL hosts, load orders, file presence, generated outputs and the checksum list (CI and locally).

16. Physical testing

Automated tests never replace physical Surface tests. Per model, the quicklist [docs/PHYSICAL-TEST-QUICKLIST.md](#) and the matrix [docs/TESTMATRIX.md](#) apply (boot, detection, routing/safety, deployment, first boot, OOBE, input, Wi-Fi, USB, Device Manager, Autopilot/Intune, reboot/shutdown). Negative physical tests: an unsupported model shows red without touching Disk 0; ARM/Snapdragon is refused; a missing archive stops safely; no step ever asks for input.

Current physical status: only **SF6 is physically validated**; SF4, SF5, SF7 and SF8 are **NOT PHYSICALLY VALIDATED** (Appendix B). A model may only be deployed in production after a successful physical test of that model on that media round.

17. Production release

A release is allowed only when all gates are green (procedure: [docs/generated/RSS_Test_and_Release_Procedure.*](#)):

1. CI green (syntax, PSScriptAnalyzer, Pester, manifest consistency, links, secret scan, no binaries);
2. media validated: [Validate-RSSMedia.ps1](#) 0 FAIL on the candidate stick;
3. physical test performed and documented per intended model;
4. known limitations documented (CHANGELOG + manifest);
5. tag (semver [vX.Y.Z](#)), checksums rebuilt ([Build-Checksums.ps1](#)), manifest release fields updated.

The distinction is mandatory: *code validated* $\neq$ *media validated* $\neq$ *physically validated* $\neq$ *production approved*.

18. Troubleshooting (evidence-based)

Method: symptom $\rightarrow$ likely cause $\rightarrow$ diagnosis $\rightarrow$ safe solution. Preserve evidence (photo of the red screen + [RSS-ADK-Deploy.log](#) or the OOBE diagnostics folder) before every action; never recover ad hoc by copying individual files back or by "repairing" a production stick on the spot.

Symptom	Likely cause	Diagnosis	Safe solution
Red screen before the countdown	preflight stop (model/SKU/CPU/media/disks/manifest)	read the error message; start of RSS-ADK-Deploy.log; Disk 0 is untouched	fix the cause (e.g. update the manifest after SKU verification), then retry
"SystemSKU is unknown"	new regional SKU variant	compare the SKU from the red screen/log with the Microsoft SystemSKU reference	verification, manifest update, revalidation, new stick
Red screen at the hash check	corrupt or outdated image/archive on the stick	re-derive the hash locally against the manifest	rebuild the stick; never adjust a hash to make the error go away
Driver archive check fails (DISM)	damaged/wrong archive	hash + dism /Get-Wim-Info + trial extraction	rebuild the Archives phase
INF count deviates	incomplete or new Microsoft package	compare the count with the MSI extraction	verify the package, manifest update, retry — never force it through

Symptom	Likely cause	Diagnosis	Safe solution
Apply-Image fails	corrupt image or USB I/O	hash; test a different USB port/stick	rebuild the media; replace the stick if it recurs
BCDBoot fails	EFI partition or applied directory incorrect	is Windows\System32\config\SYSTEM present? check the partition layout	rebuild; include the log
No keyboard/touch in WinPE	rare; boot.wim is not the ADK build or startnet fails	check Build-WinPE.status + log	rebuild the BootWim phase; external USB keyboard for diagnosis only
"Findstr is not recognized"	legacy/minimal WinPE construction	not the active production route	rebuild the stick from the ADK source
Red screen: no internal NVMe	target device is not a supported Surface, or the storage controller is in the wrong mode	verify the device model; UEFI storage setting	deploy supported models only
Red screen: multiple NVMe disks	non-standard hardware	check the configuration	the device is outside the design; do not force it
OOBE hangs at the network check	NCSI probe blocked / not registered in Autopilot / endpoint filtering	chapter 13.2; Collect-RSSOOBEDiag.ps1	fix the network/tenant side; never OOBE bypass
OOBE falls back to Wi-Fi	NCSI does not succeed within the timeout	NCSI event log, run the probe manually	involve the network team; rule out a captive portal
Autopilot "Something went wrong"	endpoints blocked or a registration/inference issue	Autopilot event log 807/815/100; check the registration in Intune	upload the hash/assign the profile; reboot into OOBE (Shift+F10, shutdown /r /t 0)
Short reboot loop at first boot	first-boot PnP processing (known phenomenon, self-recovering)	wait ≥ 10 minutes	only then investigate the Panther log
Stick full when copying	file copy instead of a block-level clone	compare partition size/logical contents	clone at block level again
Deployment restarts by itself after a successful install	USB ahead of the internal SSD in the UEFI boot order	check the boot order	remove the USB at the green screen; correct the boot order

19. Recovery

- **Preflight stop:** Disk 0 is untouched; power off the device, fix the cause, retry. No recovery needed.
- **Interruption after the wipe, before a successful apply:** the device has no working operating system; restart the procedure with the same stick (the preflight runs again and the wipe starts again — the device is already empty).
- **Corrupt stick:** rebuild via [Update-RSSMedia.ps1](#); never recover by replacing individual files. Before recovering, save the logs and note the model, SKU, failing step, time and stick used.
- **Recovery of user data:** out of scope — RSS is a wipe-and-install process; arrange an organization-level backup in advance.

20. Logging

Primary log: `<Images>\RSS-ADK-Deploy.log`, refreshed on every run. Contents: timestamp, manifest version, model/SystemSKU/CPU, media and target disk (bus, size), image and archive paths, expected INF count, hash checks, all DISM/wimlib/BCDBoot output, DriverStore counts and, as the final line, `RESULT=SUCCESS` or `RESULT=FAILED-SAFE MESSAGE=...`. The log contains no user data and stays on the stick; in incidents, copy it before rebuilding.

21. Checksums and integrity

- `checksums/SHA256SUMS.txt` covers all Git-managed files (excluding the list itself and `docs/generated`); rebuild after every change: `tools\Build-Checksums.ps1`.
- Image, archive and boot.wim hashes are in `config/sources.json` and are checked at three moments: at download (maintenance script), at stick validation and before the wipe (RSS-Deploy).
- Recipients of a stick verify at minimum: the boot.wim hash, the five install.esd hashes, the five archive hashes and `sources.json` identity.

22. Release management

See chapter 17 and the separate procedure (`RSS_Test_and_Release_Procedure`). Version numbering: semver at repository level (`vX.Y.Z`); media identity = Windows build + driver date + boot.wim hash (in the manifest and `RSS-INFO.txt`). Each released stick version gets a Git tag and a checksum list belonging to that tag.

23. Security and privacy

- No secrets, passwords, tokens, product keys, hardware hashes or serial numbers in Git or logs; CI runs gitleaks and a binary scanner.
- RSS uses only official Microsoft sources with hash and signature checks; no internet during deployment.
- Stick documentation contains no organization or customer data; the repository remains internal/proprietary without an open-source license.
- Deployment logs stay on the stick and are not collected centrally; in incidents, share them only deliberately and minimally.
- Secure Boot stays on; the boot environment contains only Microsoft-signed binaries and the BCD without test flags (validated).

24. Known limitations

- Only the five named profiles; all other hardware (including ARM/Snapdragon and the 5G variant) is deliberately out of scope.
- SF4 covers only the AMD variant; SF8 only the Intel Business edition.
- Physical validation is missing for SF4, SF5, SF7 and SF8 on the current media round.
- The SafeOS DU KB5124015 in the production media has been superseded by KB5125758; adopt the successor at the next rebuild.

- Windows 26H2 is available but is a separate image track (do not adopt it silently).
- Surface Laptop 4 driver/firmware servicing ends 2027-04-15.
- OOBЕ depends on the network (by design); an environment that blocks the NCSI/Autopilot endpoints will not let OOBЕ complete — that is a network/tenant condition, not an image defect.

Appendix A — Versions and sources

Component	Value	Source
Windows	11 25H2 build 26200.9457 (Professional, Netherlands (nl-NL))	UUP set + manifest
LCU	KB5129195 (2026-09-14)	Microsoft Update Catalog
SSU	KB5129195 bundled (KB5124007)	idem
.NET CU	KB5126052	idem
SafeOS DU	KB5124015 (media) / KB5125758 (next rebuild)	idem
ADK + WinPE add-on	10.1.26100.9457	learn.microsoft.com (adk-install)
wimlib	1.14.5	wimlib.net
Driver packages	see chapter 6	Microsoft Download Center

Appendix B — Model matrix with validation status

Profile	Model	SystemSKUs (supported)	INF	Physical status
SF4	Surface Laptop 4 (AMD)	Surface_Laptop_4_1958: 1959 · Surface_Laptop_4_1952:1953	78	NOT PHYSICALLY VALIDATED
SF5	Surface Laptop 5 (Intel, consumer + for Business)	Surface_Laptop_5_1950: 1951 · Surface_Laptop_5_for_Business_1950:1951 · Surface_Laptop_5_1979 · Surface_Laptop_5_for_Business_1979	112	NOT PHYSICALLY VALIDATED
SF6	Surface Laptop 6 for Business (Intel)	Surface_Laptop_6_for_Business_2033 · Surface_Laptop_6_for_Business_2035	116	PHYSICALLY VALIDATED (2026-09)
SF7	Surface Laptop for Business 7th Edition with Intel	Surface_Laptop_7th_Edition_With_Intel_For_Business_2107 · Surface_Laptop_7th_Edition_With_Intel_For_Business_2108	119	NOT PHYSICALLY VALIDATED
SF8	Surface Laptop for Business 8th Edition with Intel (NOT Snapdragon/ARM)	Surface_Laptop_for_Business_13_8in_8th_Ed_Intel_2107 · Surface_Laptop_for_Business_15in_8th_Ed_Intel_2108 · Surface_Laptop_for_Business_13_8in_8th_Ed_Intel_2135 · Surface_Laptop_for_Business_15in_8th_Ed_Intel_2136	120	NOT PHYSICALLY VALIDATED

N.B.: "supported" means designed and validated in software; only the status "PHYSICALLY VALIDATED" proves a successful physical deployment on that model.